



CÓDIGO BASE

SQL DESDE CERO

Guía práctica para consultar y modelar bases de datos relacionales

TINTA DIGITAL

Nivel: Principiante–Intermedio

SQL desde Cero

Guía práctica para consultar y modelar bases de datos relacionales

Colección: Código Base

Nivel: Nivel: Principiante–Intermedio

Editorial: Tinta Digital

Edición: 1.^a edición digital, 2026

Este ebook forma parte del catálogo autogenerado de Tinta Digital. Su contenido puede reproducirse con fines de estudio personal citando la fuente.

Índice

- 01 — Qué es una base de datos relacional
- 02 — SELECT: consultar datos
- 03 — Funciones de agregación y GROUP BY
- 04 — JOIN: combinar tablas
- 05 — INSERT, UPDATE y DELETE
- 06 — Diseño básico de tablas
- 07 — Subconsultas y CTE

01. Qué es una base de datos relacional

“Una tabla bien diseñada vale más que mil consultas parcheadas.”

Una base de datos relacional organiza la información en tablas compuestas por filas (registros) y columnas (campos). Cada tabla representa una entidad —clientes, pedidos, productos— y las relaciones entre tablas se expresan mediante claves.

Claves primarias y foráneas

- **Clave primaria (PK):** identifica de forma única cada fila de una tabla.
- **Clave foránea (FK):** campo que referencia la clave primaria de otra tabla, estableciendo una relación.

Por ejemplo, en una tabla pedidos, el campo cliente_id sería una clave foránea que apunta a la tabla clientes. Esta estructura evita duplicar datos y mantiene la integridad de la información.

02. SELECT: consultar datos

La instrucción SELECT es el punto de partida de casi cualquier consulta SQL.

```
SELECT nombre, email  
FROM clientes  
WHERE pais = 'España'  
ORDER BY nombre ASC;
```

Cláusulas fundamentales

- **WHERE:** filtra filas según una condición.
- **ORDER BY:** ordena el resultado (ASC o DESC).
- **LIMIT:** restringe el número de filas devueltas.
- **DISTINCT:** elimina filas duplicadas del resultado.

Operadores en WHERE

```
SELECT * FROM productos  
WHERE precio BETWEEN 10 AND 50  
AND categoria = 'libros'  
AND nombre LIKE 'Guía%';
```

03. Funciones de agregación y GROUP BY

Cuando necesitas resumir datos —totales, promedios, conteos— entran en juego las funciones de agregación combinadas con GROUP BY.

```
SELECT categoria, COUNT(*) AS total, AVG(precio) AS precio_medio
FROM productos
GROUP BY categoria
HAVING COUNT(*) > 5
ORDER BY total DESC;
```

Diferencia entre WHERE y HAVING

WHERE filtra filas antes de agrupar; HAVING filtra grupos después de aplicar la agregación. Un error habitual es intentar usar una función como COUNT() dentro de WHERE: SQL lo rechaza porque en ese punto del proceso todavía no existen los grupos.

04. JOIN: combinar tablas

Los JOIN permiten combinar filas de varias tablas relacionadas mediante sus claves.

```
SELECT pedidos.id, clientes.nombre, pedidos.total
FROM pedidos
INNER JOIN clientes ON pedidos.cliente_id = clientes.id
WHERE pedidos.total > 100;
```

Tipos de JOIN

- **INNER JOIN:** solo filas que coinciden en ambas tablas.
- **LEFT JOIN:** todas las filas de la tabla izquierda, aunque no haya coincidencia en la derecha.
- **RIGHT JOIN:** equivalente, pero priorizando la tabla derecha.
- **FULL OUTER JOIN:** todas las filas de ambas tablas, coincidan o no.

Una forma útil de visualizarlo: INNER JOIN es la intersección de dos conjuntos; LEFT JOIN es el conjunto izquierdo completo más la intersección.

05. INSERT, UPDATE y DELETE

Además de consultar, SQL permite modificar los datos.

```
INSERT INTO clientes (nombre, email, pais)
VALUES ('Marta López', 'marta@correo.com', 'España');
```

```
UPDATE clientes
SET pais = 'Portugal'
WHERE id = 12;
```

```
DELETE FROM clientes
WHERE id = 12;
```

UPDATE y DELETE sin una cláusula WHERE afectan a todas las filas de la tabla. Es el error más costoso —y más común— en SQL: conviene probar siempre primero un SELECT con la misma condición para verificar qué filas se van a ver afectadas.

06. Diseño básico de tablas

Un buen diseño evita redundancia y facilita el mantenimiento. La normalización organiza los datos en tablas separadas para que cada dato se almacene una única vez.

```
CREATE TABLE clientes (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nombre TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL  
);  
  
CREATE TABLE pedidos (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  cliente_id INTEGER,  
  total DECIMAL(10,2),  
  FOREIGN KEY (cliente_id) REFERENCES clientes(id)  
);
```

Buenas prácticas al modelar

- Cada tabla representa una única entidad.
- Usa NOT NULL en los campos obligatorios.
- Define UNIQUE en campos que no deben repetirse, como el email.
- Nombra las tablas en plural y los campos en singular, de forma consistente.

07. Subconsultas y CTE

Una subconsulta es una consulta dentro de otra. Las CTE (Common Table Expressions), definidas con WITH, hacen lo mismo pero de forma más legible cuando la lógica es compleja.

```
WITH ventas_por_cliente AS (  
  SELECT cliente_id, SUM(total) AS total_gastado  
  FROM pedidos  
  GROUP BY cliente_id  
)  
SELECT clientes.nombre, ventas_por_cliente.total_gastado  
FROM ventas_por_cliente  
JOIN clientes ON clientes.id = ventas_por_cliente.cliente_id  
WHERE total_gastado > 500;
```

Las CTE son especialmente útiles cuando una misma subconsulta se reutiliza varias veces, o cuando el encadenamiento de subconsultas anidadas empieza a dificultar la lectura.